



Memoria tecnica

LoveCode

Pablo Greus

26/04/2026



Indice

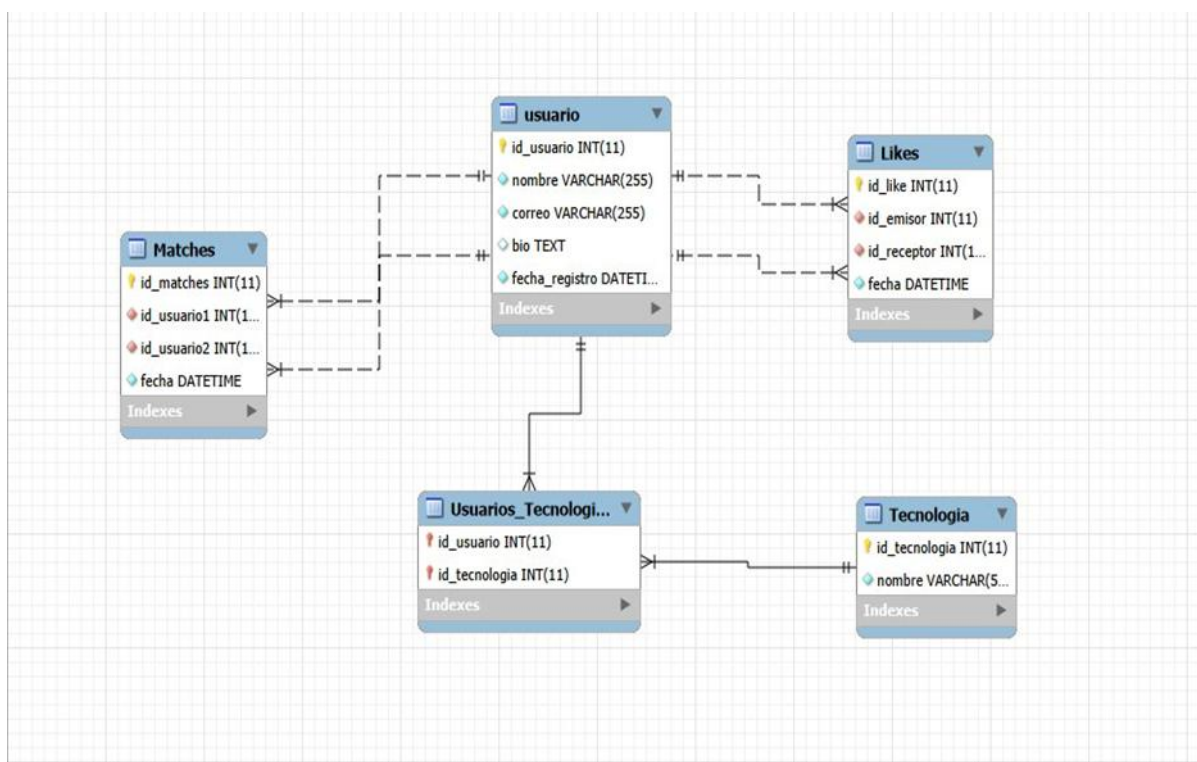
Descripcion	3
Bases de datos	3
Tablas	3
Usuario.....	3
Tecnologias.....	4
Usuarios_Tecnologias.....	4
Likes.....	4
Matches	4
Backend Java.	4
Clases	4
FrontEnd.....	5
Problemas y soluciones.....	5

Descripcion

CodeCrush es una aplicación web de citas diseñada específicamente para desarrolladores de software. Al estilo de Tinder, permite a los usuarios encontrar otros programadores con tecnologías y gustos compatibles, dar likes, y generar matches cuando el interés es mutuo.

El proyecto integra tres capas tecnológicas: una base de datos relacional MariaDB, un backend en Java con acceso mediante JDBC, y un frontend HTML/CSS estático.

Bases de datos



Tablas

Usuario

id_usuario, INT PK AUTOINCREMENT, Identificador único del usuario

nombre, VARCHAR(255), Nombre del usuario

correo, VARCHAR(255) UNIQUE, Correo electrónico, no puede repetirse

bio, TEXT, Descripción personal, pequeña descripción para darse a conocer.



fecha_registro, DATETIME DEFAULT NOW(), Fecha de alta automática

Tecnologias

id_tecnologia, INT PK AUTO, Identificador único

nombre, VARCHAR(50) UNIQUE, Nombre del lenguaje.

Usuarios_Tecnologias

Esta tabla implementa la relacion N:M entre usuario y tecnologia.

Likes

Id_Like, INT PK AUTO, identificador del like.

Id_emisor, INT FK, usuario que da el like.

id_receptor, INT FK, Usuario que recibe el like

fecha, DATETIME DEFAULT NOW(), Momento del like

Matches

id_matches, INT PK AUTO, Identificador del match

id_usuario1, INT FK, Primer usuario del par

id_usuario2, INT FK, Segundo usuario del par

fecha, DATETIME DEFAULT NOW(), Momento en que se produjo el match.

Backend Java.

Clases

Variables.java. Centraliza la URL de conexión JDBC, el usuario y la contraseña de MariaDB.

Usuario.java. Clase que modela la entidad usuario con sus atributos y getters/setters.

UsuarioDAO. Java Implementa las operaciones CRUD sobre la tabla usuario: insertar, listar, buscar por id y eliminar. Incluye validación de duplicados.

Like.java. Clase para la entidad Like (Por terminar).

LikeDAO.java DAO para la tabla Likes (Por terminar).

Mach.java. Clase para la entidad Match (Por terminar).



App.java. Clase de prueba de conexión: ejecuta SELECT 1 para verificar que JDBC conecta con MariaDB.

Main.java. Punto de entrada principal: lista todos los usuarios de la BD por consola.

Conexion JDBC

La conexión a la base de datos se realiza mediante el driver MariaDB JDBC. Los parámetros están centralizados en Variables.java:

```
package com.example.BackEnd;

public class Variables {
    public static String url = "jdbc:mariadb://192.168.16.3:3306/LoveCode";
    public static String user = "admin";
    public static String password = "admin";
}
```

El servidor MariaDB corre en una máquina virtual con IP 192.168.16.3, escuchando en el puerto por defecto 3306.

FrontEnd

3 paginas desarrolladas.

Index.html que seria el 'inicio' en la cual tiene una bienvenida con el logo y dos botones para acceso a login y registro.

Login.html en la cual tendremos un formulario de inicio de sesion con usuario y contraseña.

Registrar.html donde tendremos un formulario de alta xon nombre, correo, contraseña, bio y selección de tecnologías.

Y un Styles.css donde estaran todos los estilos.

Problemas y soluciones

Nombre de tabla incorrecto: "usuarios" en lugar de "usuario"

Ramas de git y comits incorrectos. Nombres que no tienen sentido.

Al hacer la base de datos se olvida de hacer la FK y toca dropear toda la base y volverla a hacer desde el principio.

Permisos de los usuarios incorrectos toco borrar todos los usuarios y volverlos a hacer.

